

Dr. Sophie Bots (she/her)

Assistant Professor

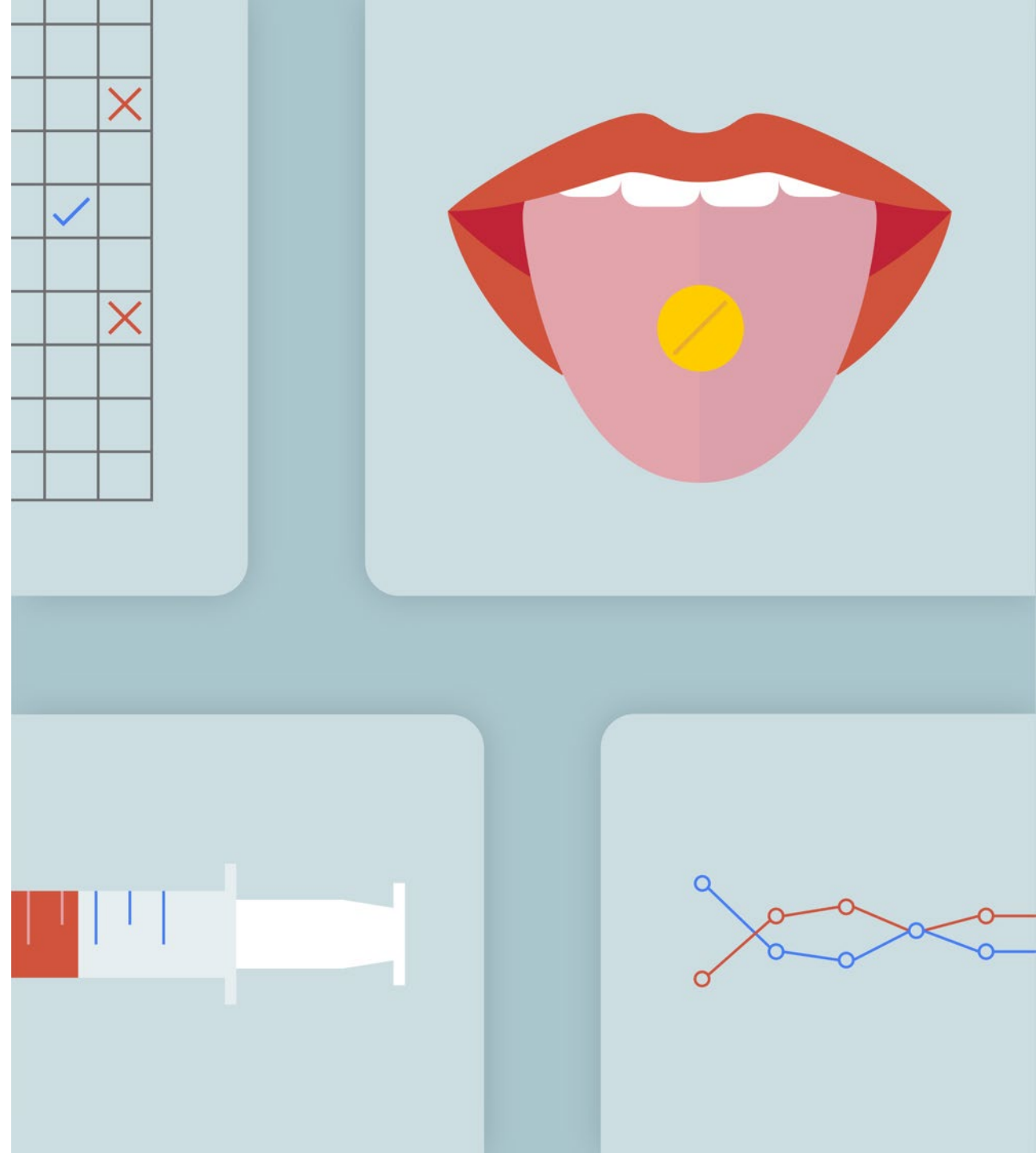
R crash course

session 2



Utrecht University

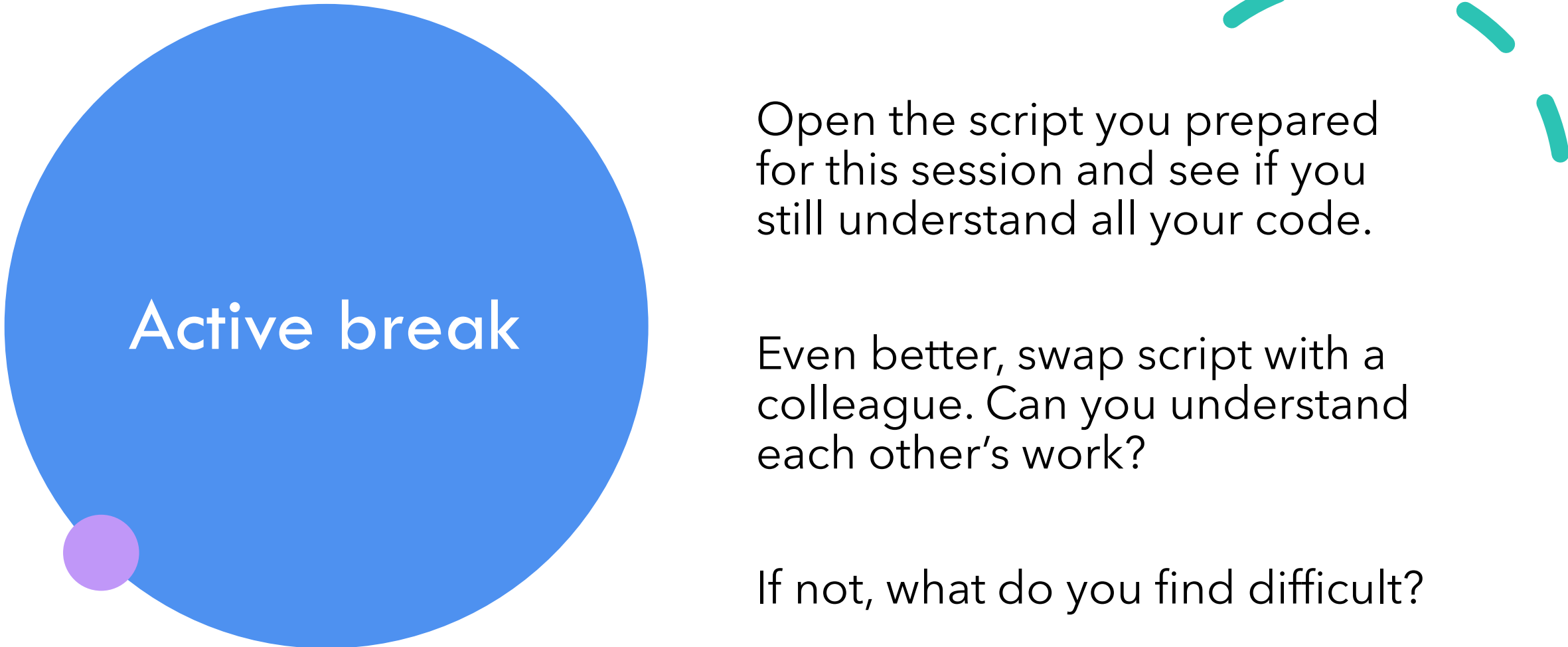
Pharmacoepidemiology
and Clinical Pharmacology





Preparation for session 2

walkthrough



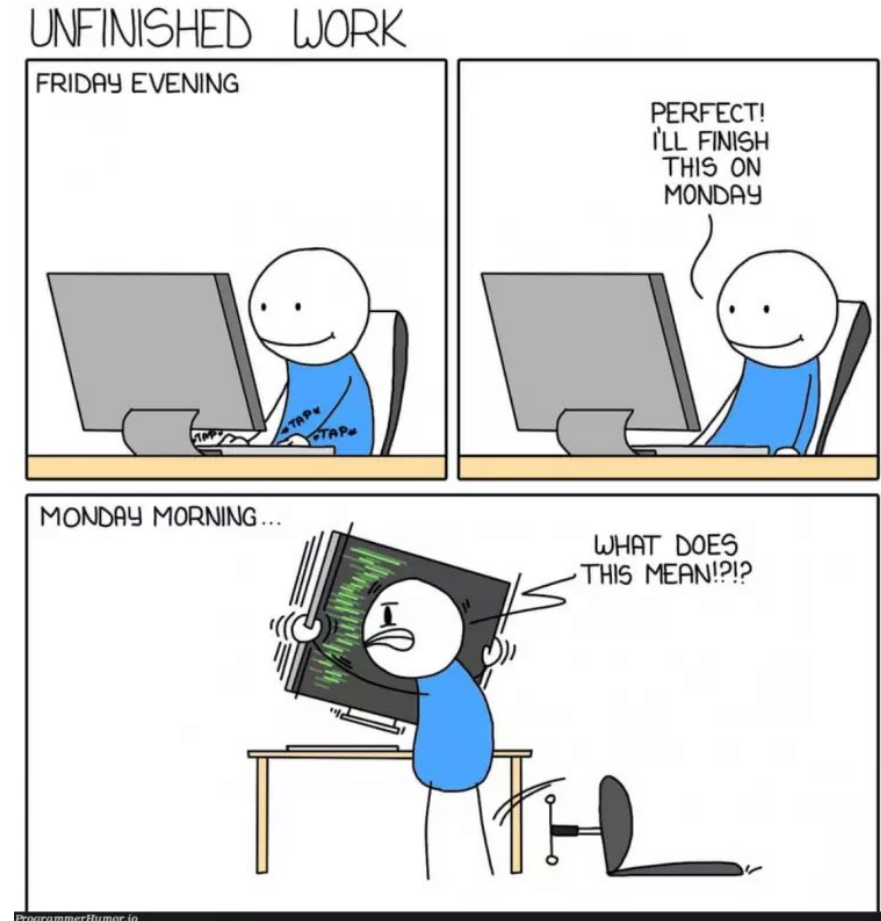
Active break

Open the script you prepared for this session and see if you still understand all your code.

Even better, swap script with a colleague. Can you understand each other's work?

If not, what do you find difficult?

Treat others the way you want to be treated ...



Take home messages on good (R) coding practice

- Follow good naming practice
- Annotate your script using comments
- Follow code structure conventions
- Google is your best friend

```
i_use_snake_case  
otherPeopleUseCamelCase  
some.people.use.periods  
And_aFew.People.RENOUNCEconvention
```

```
# Strive for  
flights |>  
  filter(!is.na(arr_delay), !is.na(tailnum)) |>  
  count(dest)  
  
# Avoid  
flights|>filter(!is.na(arr_delay), !is.na(tailnum))|>count(dest)
```

A note on coding and generative AI

Generative AI models can help you with your coding, and are often useful

Treat GenAI as a colleague, who wants to help you but

- may need quite detailed explanation to understand your question
- consumes quite a bit of energy

1. ChatGPT consumes over half a million kilowatts of electricity each day, an amount staggering enough to service about two hundred million requests.
2. ChatGPT's daily power usage is nearly equal to 180,000 U.S. households, each using about twenty-nine kilowatts.
3. A single ChatGPT conversation uses about fifty centilitres of water, equivalent to one plastic bottle.

... and can be wrong!

*"In conclusion, we systematically studied package hallucinations in code generation LLMs, including both commercial and open-source models. Our comprehensive analysis revealed that **19.7%** of generated packages are fictitious."* (Spracklen et al., 2024)

Editorial | [Open access](#) | Published: 19 August 2023

ChatGPT: these are not hallucinations – they're fabrications and falsifications

[Robin Emsley](#) 

[Schizophrenia](#) 9, Article number: 52 (2023) | [Cite this article](#)

[Emsley R., Schizophr, 2023](#)
[Spracklen et al., Arxiv, 2024](#)



So then why this course?

- a simple Google search is more energy-efficient than asking GenAI – know how and where to find answers to common questions within the R community and stackoverflow
- errors are easier to spot when you know how R code works – learn the basics of coding so you can spot simple errors early

Hallucinations in code are the least dangerous form of LLM mistakes

A surprisingly common complaint I see from developers who have tried using LLMs for code is that they encountered a hallucination—usually the LLM inventing a method or even a full software library that doesn't exist—and it crashed their confidence in LLMs as a tool for writing code. How could anyone productively use these things if they invent methods that don't exist?

Hallucinations in code **are the least harmful hallucinations you can encounter from a model.**

(When I talk about *hallucinations* here I mean instances where an LLM invents a completely untrue fact, or in this case outputs code references which don't exist at all. I see these as a separate issue from bugs and other mistakes, which are the topic of the rest of this post.)

The real risk from using LLMs for code is that they'll make mistakes that *aren't* instantly caught by the language compiler or interpreter. And these happen *all the time!*

On spotting errors

Hallucinations in code are the least dangerous form of LLM mistakes

A surprisingly common complaint LLMs for code is that they encourage inventing a method or even a full crash that crashed their confidence in LLMs productively use these things if they

Hallucinations in code **are the least dangerous** encounter from a model.

(When I talk about *hallucinations* a completely untrue fact, or in this case, they don't exist at all. I see these as a separate category of errors which are the topic of the rest of the talk)

The real risk from using LLMs for code is that they'll make mistakes that *aren't* instantly caught by the language compiler or interpreter. And these happen *all the time*!

Package Hallucinations: How LLMs Can Invent Vulnerabilities

A Comprehensive Study that Sheds Light on a Systemic and Persistent Threat

June 17, 2025

RESEARCH

Authors: [Joseph Spracklen](#), [Raveen Wijewickrama](#), [A.H.M. Nazmus Sakib](#), [Anindya Maiti](#), [Bimal Viswanath](#), [Murtuza Jadliwala](#)

Article shepherded by: Rik Farrow



So then why this course?

- a simple Google search is more energy-efficient than asking GenAI
 - know how and where to find answers to common questions within the R community and stackoverflow
 - errors are easier to spot when you know how R code works – learn the basics of coding so you can spot simple errors early
 - explaining yourself to others is a skill – practice troubleshooting so you can give GenAI better instructions
 - coding is a very useful skill to have, being able to perform the basics will help you tremendously during research assignments and internships
- 

Learning objectives covered so far

1. Perform basic coding tasks in RStudio like creating variables, and indexing data frames
- 2. Apply good workflow practices for coding, scripting, and getting help (preparation)**
3. Explain the rationale behind the tidy data framework and why this benefits data analysis
4. Transform datasets with the *dplyr* package
5. Visually explore datasets with *ggplot2* and interpret the results (exploratory analysis)
6. Perform descriptive and statistical analyses and interpret the results

Learning goals for today

1. Perform basic coding tasks in RStudio like creating variables, and indexing data frames
2. Apply good workflow practices for coding, scripting, and getting help
3. Explain the rationale behind the tidy data framework and why this benefits data analysis
- 4. Transform datasets with the *dplyr* package**
5. Visually explore datasets with *ggplot2* and interpret the results (exploratory analysis)
6. Perform descriptive and statistical analyses and interpret the results

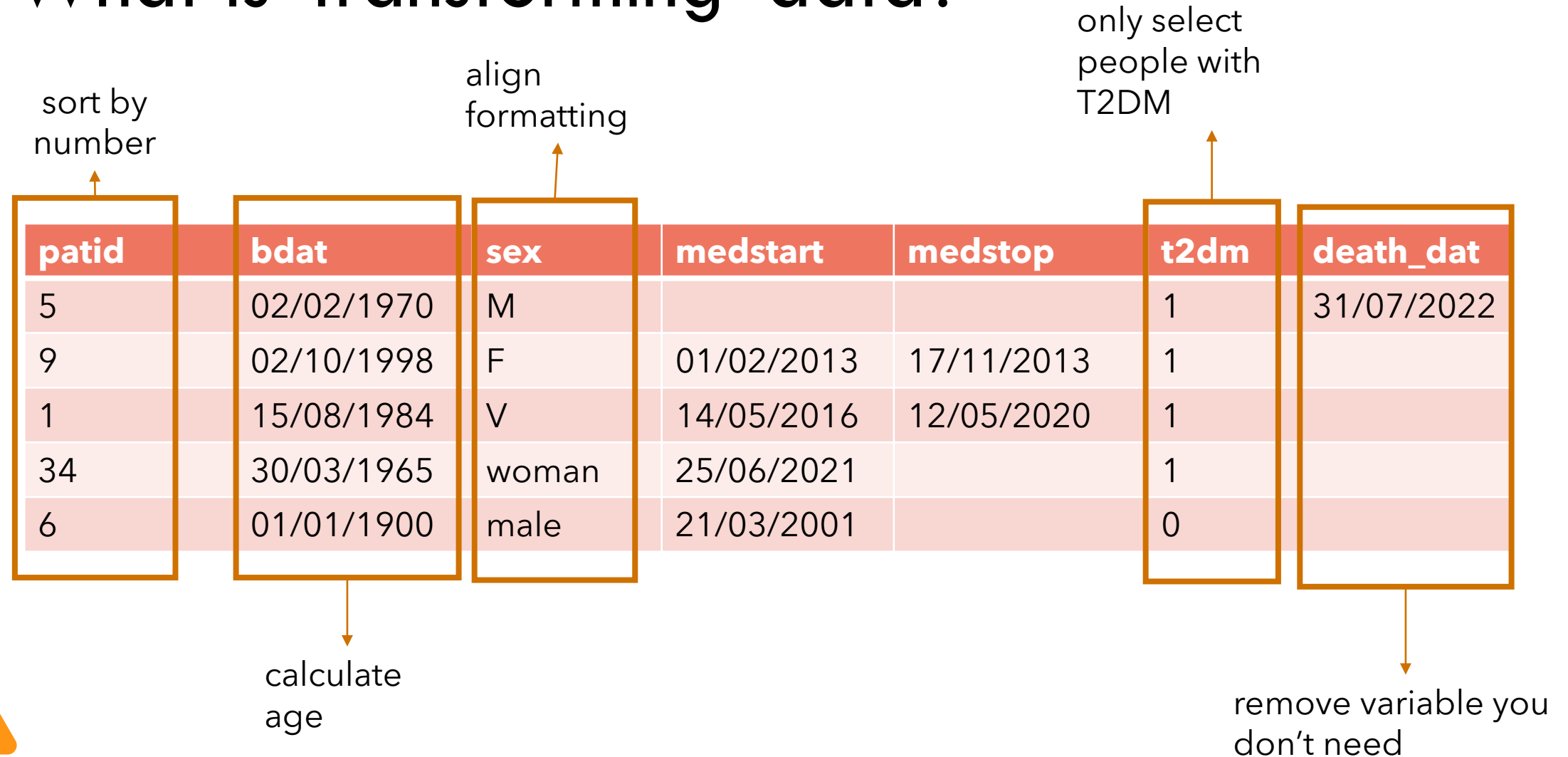


Transform datasets with *dplyr*

Part 1
learning goal 4



What is 'transforming' data?



Row or column?

sort by number		align formatting		only select people with T2DM		
patid	bdat	sex	medstart	medstop	t2dm	death_dat
5	02/02/1970	M			1	31/07/2022
9	02/10/1998	F	01/02/2013	17/11/2013	1	
1	15/08/1984	V	14/05/2016	12/05/2020	1	
34	30/03/1965	woman	25/06/2021		1	
6	01/01/1900	male	21/03/2001		0	

calculate age

remove variable you don't need

The *dplyr* package



Collection of functions with the same structure:
function(data frame, variable names...)

Each function name is a verb (the transformation performed)

- *filter()*
- *arrange()*
- *select()*
- *mutate()*



Row: exclude observations with *filter()*

filter(dataset, variable name & value ...)

```
new_data <- filter(old_data,  
  bmi < 60)
```

What do you want?

- specific value (==)
- relative value (</>)
- not this value (!)

Missing values will be deleted

distinct() is a special version of filter that removes duplicates

==	equal
!=	not equal
<	less than
<=	less than or equal
>	greater than
>=	greater than or equal
	or
!	not
%in%	in the set



Row: sort observations with `arrange()`

`arrange(dataset, variable name, ...)`

Options

- sort based on multiple variables
- in descending order

```
new_data <- arrange(old_data,  
                     patid)  
  
new_data <- arrange(old_data,  
                     desc(patid))
```

Missing values end up at the end of the dataset

Column: pick variables with `select()`

`select(dataset, variable name, ...)`

Useful shortcuts

- `:` to select all columns between A&B
- `-` to remove a column
- helper functions
 - `starts_with`, `ends_with`, `contains`
- `?select()` for all options

`select(df, A:B)`

`select(df, -A)`

`select(df, contains("dat"))`



Column: create variables with *mutate()*

mutate(dataset, newvariable = f(oldvariables))

What can you put as *f(oldvariables)*?

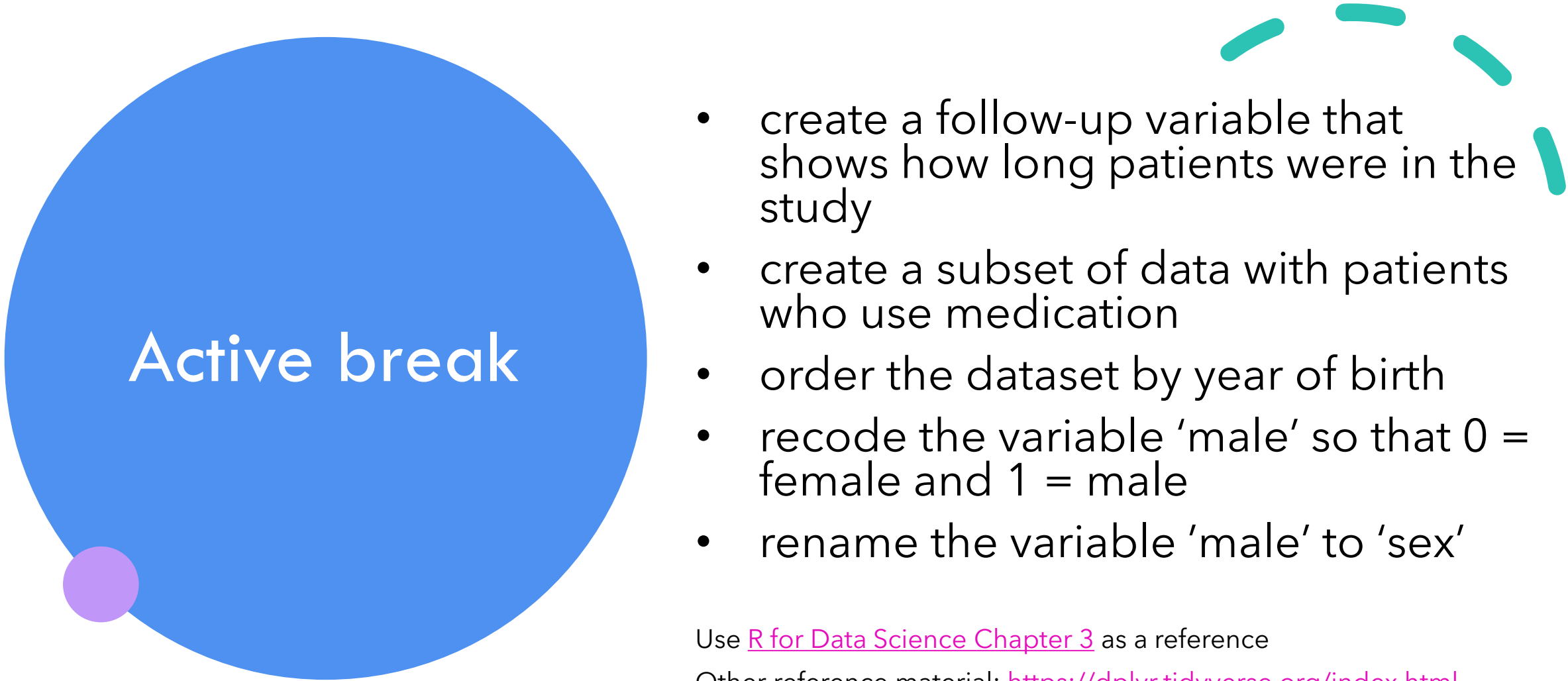
```
new_data <- mutate(old_data,  
  bmi = weight/height ^2)
```

- simple calculations: +, -, /, *, ^ *mutate*(df, newvar = var1+var2)
- comparisons: <, > *mutate*(df, newvar = var1<var2)

Column: other relevant functions



- *rename()* – give variables a new name
- *relocate()* – change order of variables
- *mutate_[at/if]()* – create multiple variables at once



Active break

- create a follow-up variable that shows how long patients were in the study
- create a subset of data with patients who use medication
- order the dataset by year of birth
- recode the variable 'male' so that 0 = female and 1 = male
- rename the variable 'male' to 'sex'

Use [R for Data Science Chapter 3](#) as a reference

Other reference material: <https://dplyr.tidyverse.org/index.html>

and *R for Data Science* chapters 12-19 for a deep dive

Anyone run into this problem?

```
flights1 <- filter(flights, dest == "IAH")  
flights2 <- mutate(flights1, speed = distance / air_time * 60)  
flights3 <- select(flights2, year:day, dep_time, carrier, flight, speed)  
arrange(flights3, desc(speed))
```



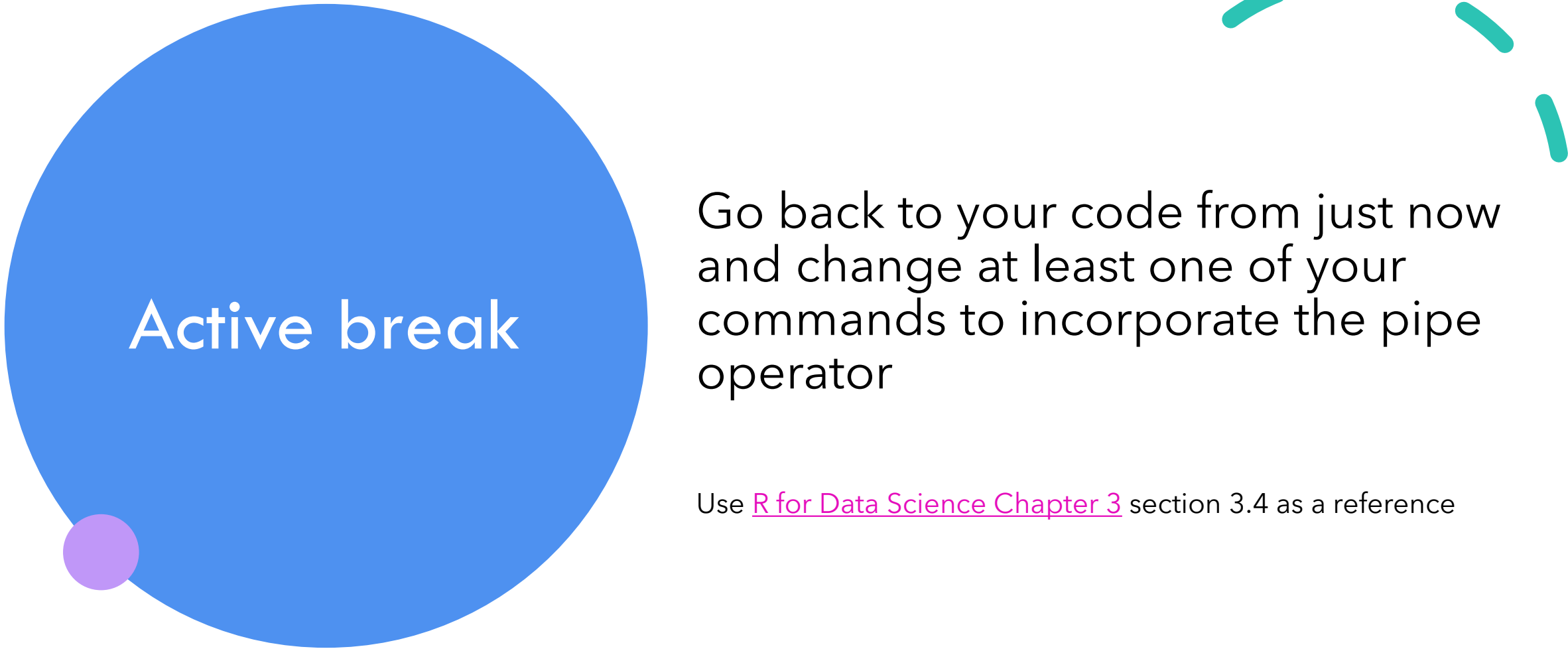
Introducing the pipe operator



```
flights1 <- filter(flights, dest == "IAH")
flights2 <- mutate(flights1, speed = distance / air_time * 60)
flights3 <- select(flights2, year:day, dep_time, carrier, flight, speed)
arrange(flights3, desc(speed))
```

```
flights |>
```

```
  filter(dest == "IAH") |>
  mutate(speed = distance / air_time * 60) |>
  select(year:day, dep_time, carrier, flight, speed) |>
  arrange(desc(speed))
```



Active break

Go back to your code from just now and change at least one of your commands to incorporate the pipe operator

Use [R for Data Science Chapter 3](#) section 3.4 as a reference

What we covered today

1. Perform basic coding tasks in RStudio like creating variables, and indexing data frames
2. Apply good workflow practices for coding, scripting, and getting help
3. Explain the rationale behind the tidy data framework and why this benefits data analysis
- 4. Transform datasets with the *dplyr* package**
5. Visually explore datasets with *ggplot2* and interpret the results (exploratory analysis)
6. Perform descriptive and statistical analyses and interpret the results

Revise: check if you understood everything

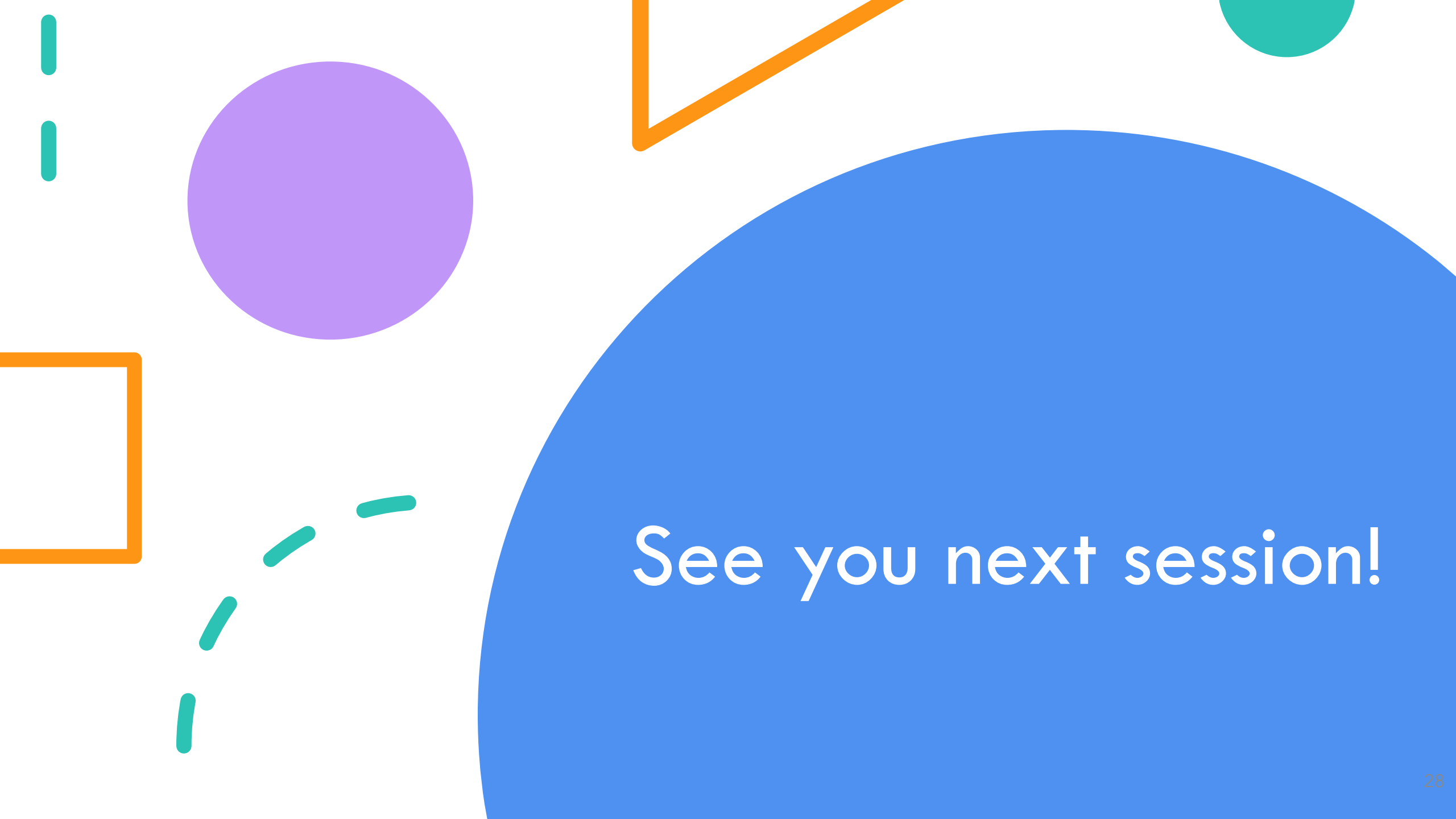
R for Data Science

- Chapter 3 exercises

Introduction to R for health data analysis

- Chapter 5

See course outline for additional revision materials and other optional further reading

The background features a large blue semi-circle on the right side. On the left, there are several decorative elements: a purple circle, an orange square outline, an orange L-shaped line, a teal circle, and several teal dashed lines of varying lengths and orientations.

See you next session!